

CBUS: Modelling and Solution Approaches for the Capacitated Bus Routing Problem

Group 20 Report

Hanoi University of Science and Technology
School of Information and Technology

July 1, 2026

Original Problem Statement

There are n passengers $1, 2, \dots, n$. Passenger i wants to travel from point i to point $i + n$, for $i = 1, 2, \dots, n$.

The bus is located at point 0 and has k places for transporting passengers, meaning that at any time there are at most k passengers on the bus.

Given the distance matrix c , where $c(i, j)$ is the traveling distance from point i to point j for $i, j = 0, 1, \dots, 2n$, compute the shortest route.



Problem setting

The CBUS problem models a single capacitated vehicle that must pick up and deliver passengers while respecting route order and capacity constraints.

- ▶ Each passenger i has pickup node i and delivery node $i + n$.
- ▶ The bus starts and ends at depot 0.
- ▶ Every service node must be visited exactly once.
- ▶ The objective is to minimize total travel distance.

Team Responsibilities

Member	Student ID	Main responsibility
Nguyen Hien Minh	202516670	Branch-and-bound solutions
Le Tri Cuong	202516646	Leader, OR-Tools SCIP/ILP, benchmarks, report, slides
Nguyen Vu Hoang Anh	202516641	OR-Tools CP-SAT and standard mathematical model
Dinh Khanh Nam	202516672	Greedy, local search, random walk, tabu search

Formal Problem Definition

- ▶ Nodes: $\mathcal{V} = \{0, 1, \dots, 2n\}$.
- ▶ Pickups: $P = \{1, \dots, n\}$.
- ▶ Deliveries: $D = \{n + 1, \dots, 2n\}$.
- ▶ Capacity: k passengers.
- ▶ Distance matrix: c_{ij} .

Route objective

$$\min c_{0,x_1} + \sum_{t=1}^{2n-1} c_{x_t,x_{t+1}} + c_{x_{2n},0}$$

Load change

$$q_v = \begin{cases} 1, & v \in P, \\ -1, & v \in D, \\ 0, & v = 0. \end{cases}$$

Core Feasibility Constraints

1. **Visit-once:** every pickup and delivery node appears exactly once.
2. **Precedence:** pickup must occur before delivery:

$$\pi_i < \pi_{i+n}, \quad \forall i \in P.$$

3. **Capacity:** bus load must remain valid:

$$l_t = l_{t-1} + q_{s_t}, \quad 0 \leq l_t \leq k.$$

4. **Depot consistency:** $s_0 = s_{2n+1} = 0$ and $l_0 = l_{2n+1} = 0$.

Exact methods

- ▶ Branch and Bound
- ▶ CP-SAT model
- ▶ Arc-based ILP/SCIP
- ▶ Sequence-based ILP comparison

Approximate methods

- ▶ Greedy nearest-neighbor construction
- ▶ Swap-move local search
- ▶ Random walk destroy-and-repair
- ▶ Dynamic tabu search

Main trade-off

Exact methods prove optimality on small instances; metaheuristics scale to large instances and return strong feasible solutions under strict time limits.

Branch and Bound: Search Tree Structure

Root: depot 0, empty passenger load, no service node visited

↓ expand by feasible node choices

Level t : route prefix $(x_1, \dots, x_t)$, current cost, current load

↓ filter candidate nodes

Feasibility: no duplicate visit, pickup before delivery, load within $[0, k]$

↓ pruning decision

Bound: keep branch only if optimistic cost can beat z_{best}

↓ termination

Leaf: complete route updates incumbent; timeout returns best incumbent

Branch and Bound: Core Recursive Routine

```
def Try(step):
    global load, current_distance, z_best, x_best
    global visited, x, n, c, c_min

    for v in range(1, 2 * n + 1):
        if check(v):
            x[step] = v
            visited[v] = True
            current_distance += c[x[step - 1]][v]
            load += 1 if v <= n else -1

            lower_bound = current_distance + (2*n-step+1)*c_min

            if lower_bound < z_best:
                if step == 2 * n:
                    total_distance = current_distance + c[x[2*n]][0]
                    if total_distance < z_best:
                        z_best = total_distance
                        x_best = list(x)
                else:
                    Try(step + 1)

            visited[v] = False
            current_distance -= c[x[step - 1]][v]
            load -= 1 if v <= n else -1
```

Commentary

The Try(step) routine fixes one more route node, checks the lower bound, then expands or prunes the branch.

- **Branching:** try every feasible node v .
- **State update:** update route, visited flags, distance, and load.
- **Bounding:** expand only if the lower bound beats z_{best} .
- **Incumbent:** update the best solution when a complete route is found.
- **Backtracking:** restore state exactly before trying another branch.

Branch and Bound: Implementation Keys

State maintained during recursion

- ▶ Route prefix and current last node.
- ▶ Boolean visited marker for each service node.
- ▶ Current bus load after the latest visit.
- ▶ Current distance accumulated so far.

Pruning and safety controls

- ▶ Reject the delivery node if the pickup is not visited.
- ▶ Reject the pickup if the load would exceed capacity k .
- ▶ Use c_{min} to estimate the remaining minimum distance.
- ▶ Enforce time limit to avoid non-returning search.

Presentation point

This method is exact when it finishes, but the search tree grows exponentially; therefore, it is mainly used as a correctness reference for small instances.

CP-SAT: Variables and Channeling Design

Main idea

- ▶ **point**[t]: node visited at route step t .
- ▶ **index**[v]: step where node v is visited.
- ▶ Two views describe the same route sequence.
- ▶ Channeling links both views for propagation.

Why it helps

Precedence becomes simple:

$index[i] < index[i + n]$, while **AllDifferent** on **point** enforces visit-once.

Route view: **point**[t] = node at step t

step t	0	1	2	3	...	$2n + 1$
point [t]	0	n_1	n_2	n_3	...	0

Index view: **index**[v] = step where node v is visited

node v	0	1	2	...	$2n$
index [v]	0	a	b	...	t

Channeling rule: if $point[t] = v$, then $index[v] = t$.

CP-SAT: Sequence Variables and Channeling

```
customer = [model.NewIntVar(0, k, ...)]
point = [model.NewIntVar(0, total_nodes-1, ...)]
index = {i: model.NewIntVar(1, total_nodes-1, ...)}

model.Add(point[0] == 0)
model.Add(point[total_nodes] == 0)
model.AddAllDifferent(point[1:total_nodes])
```

Design breakthrough

The CP model keeps two synchronized views of the same route:
position-to-node through `extttpoint[t]`
and node-to-position through
`extttindex[v]`. This makes precedence
constraints cheap and explicit.

- ▶ `point[t]` tells which node is visited at step t .
- ▶ `index[v]` tells when node v is visited.
- ▶ `AllDifferent` enforces exact visit-once structure.

CP-SAT: Propagation for Precedence, Load, and Cost

```
for i in range(1, n + 1):
    model.Add(index[i] < index[i+n])

model.Add(customer[i] == customer[i-1] + 1).OnlyEnforceIf(pickup)
model.Add(customer[i] == customer[i-1] - 1).OnlyEnforceIf(delivery
)
model.AddElement(matrix_idx, flat_matrix, step_dist[i])
step_dist = [model.NewIntVar(0, 1000000, f'step_dist_{i}') for i
              in range(2*n + 1)]
```

Why this is powerful

The solver propagates logical consequences before search: impossible orders, overload states, and inconsistent distance assignments are removed from domains before branching.

- ▶ Reified pickup/delivery booleans drive load transitions.
- ▶ Flattened matrix enables direct distance lookup.
- ▶ Objective is simply $\sum step_dist$.

Model configuration

- ▶ Sequence length: $2n + 2$ including depot start and end.
- ▶ Domain: service nodes in $\{1, \dots, 2n\}$.
- ▶ Load domain: integer range $[0, k]$ at each step.
- ▶ Objective: sum of all step distances.

Solver configuration

- ▶ Local time limit: 30 seconds per instance.
- ▶ Best for small and medium validation tests.
- ▶ Stronger than manual search due to propagation.
- ▶ Still limited by combinatorial route ordering.

Routing skeleton

- ▶ Select route decisions using binary variables.
- ▶ Enforce exactly one predecessor and successor.
- ▶ Start and finish at depot 0.
- ▶ Prevent repeated or missing service nodes.

Objective structure

- ▶ Minimize total selected travel distance.
- ▶ Distance coefficient comes directly from c_{ij} .

Validity structure

- ▶ Time/order variables remove subtours.
- ▶ Pickup time must be before delivery time.
- ▶ Load variables track passengers on the bus.
- ▶ Big- M constraints activate transitions only on selected arcs.

Key point

ILP separates the route skeleton from feasibility propagation, then links them through Big- M implications.

ILP: Constraint Activation Flow

Choose arc: binary variable decides whether the bus travels from i to j



If arc is active: enforce $t_j \geq t_i + 1$ and update load by q_j



If arc is inactive: Big- M relaxes the corresponding implication



Global route: degree constraints connect all active arcs into one feasible tour



Optimization: SCIP branches on binary decisions and improves the incumbent

ILP Models: Arc-Based vs. Sequence-Based

ILP-01: Arc-based network flow

- ▶ Binary decision: choose directed arc (i, j) .
- ▶ Degree constraints enforce one predecessor and successor.
- ▶ Time variables remove subtours and impose order.
- ▶ Load variables enforce capacity along active arcs.
- ▶ More compatible with SCIP branch-and-cut.

ILP-02: Sequence-based comparison

- ▶ Binary decision: assign node v to route step s .
- ▶ Position variables express pickup–delivery order.
- ▶ Distance requires additional linearization variables.
- ▶ Big- M constraints weaken the relaxation.
- ▶ Included to compare modelling paradigms.

Shared Feasibility Kernel for All Heuristics

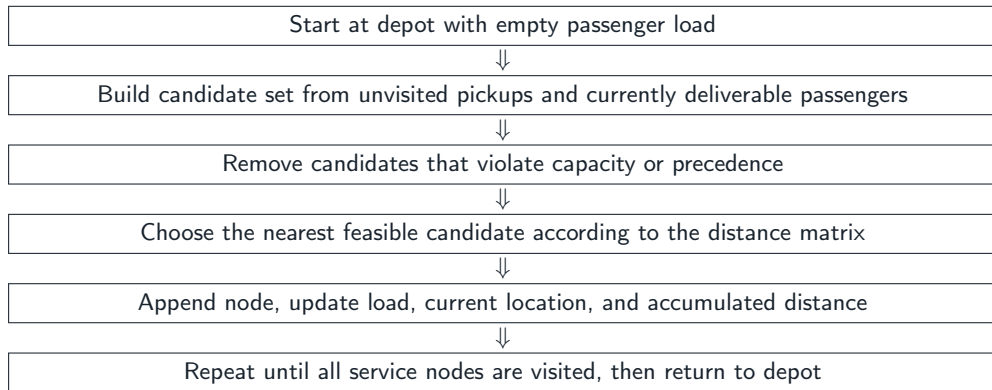
What every heuristic must preserve

- ▶ Each pickup node appears exactly once.
- ▶ Each delivery node appears exactly once.
- ▶ Pickup i is before delivery $i + n$.
- ▶ Load never becomes negative or exceeds k .

Why this kernel is important

- ▶ Allows fast comparison between methods.
- ▶ Prevents invalid local moves from entering search.
- ▶ Makes Greedy, Swap-move, Random Walk, and Tabu consistent.
- ▶ Separates route generation from feasibility checking.

Greedy Construction: Feasible Seed Generator



Greedy Construction: Key Settings and Behavior

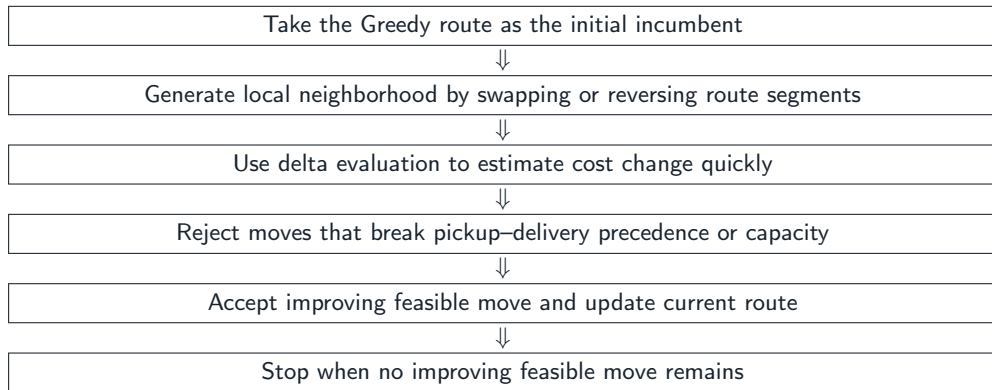
Configuration keys

- ▶ Candidate priority: minimum immediate travel distance.
- ▶ Capacity rule: pickups allowed only if load $< k$.
- ▶ Delivery rule: delivery allowed only for onboard passengers.
- ▶ Tie behavior: deterministic by candidate order.

Strengths and weaknesses

- ▶ Very fast and always produces a feasible baseline.
- ▶ Useful as initial solution for all later methods.
- ▶ Myopic decisions can create expensive future detours.
- ▶ No mechanism to revise earlier choices.

Swap-move Local Search: Exploitation Around Greedy



Swap-move Local Search: Key Settings and Behavior

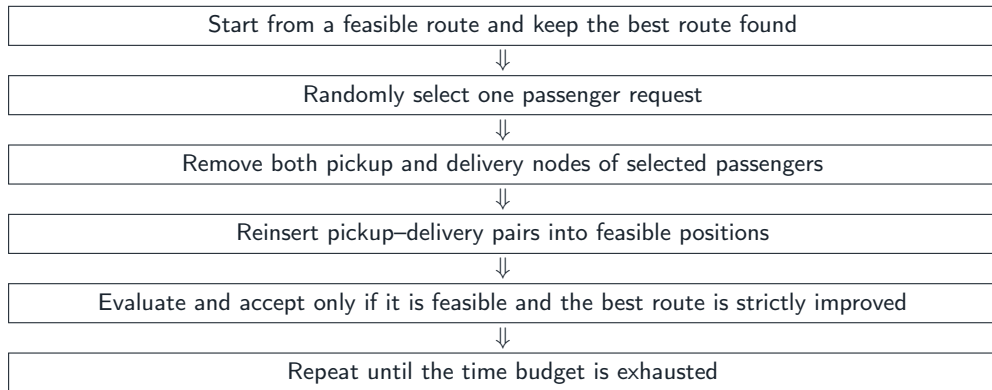
Configuration keys

- ▶ Initial route: Greedy feasible solution.
- ▶ Neighborhood: feasible pair swap or Swap-move style segment change.
- ▶ Acceptance: improving moves only.
- ▶ Evaluation: incremental cost difference when possible.

Strengths and weaknesses

- ▶ Quickly removes obvious route inefficiencies.
- ▶ Very low runtime overhead on local benchmarks.
- ▶ Can become trapped in the first local optimum.
- ▶ Needs diversification to work well on large instances.

Random Walk: Diversification



Random Walk: Key Settings and Behavior

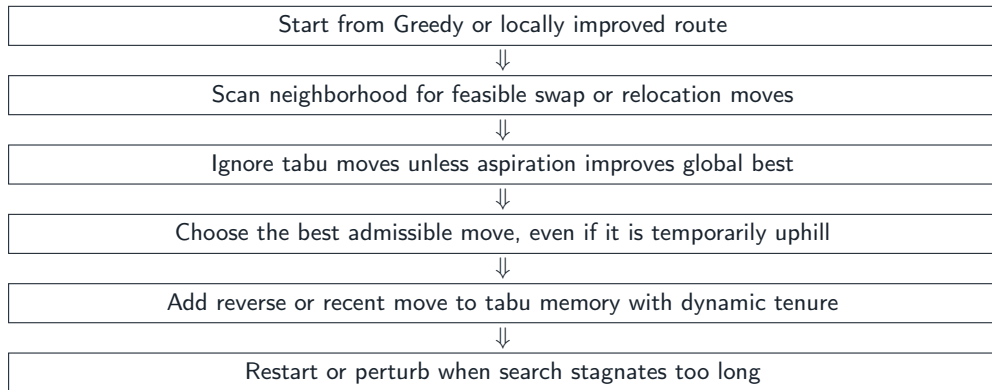
Configuration keys

- ▶ Move unit: passenger pair, not individual node only.
- ▶ Repair rule: pickup inserted before matching delivery.
- ▶ Acceptance: stochastic feasible reconstruction.
- ▶ Time control: returns best route before hard limit.

Strengths and weaknesses

- ▶ Escapes local optima through large structural changes.
- ▶ Explores many topologically different routes.
- ▶ Less predictable than deterministic local search.
- ▶ Strong performance on large hidden judge instances.

Tabu Search: Memory-Guided Neighborhood Search



Tabu Search: Key Settings and Behavior

Configuration keys

- ▶ Tabu tenure: scales with instance size $\sqrt{n}$.
- ▶ Aspiration: override tabu if global best improves.
- ▶ Stagnation rule: perturb after many non-improving iterations.
- ▶ Memory cleanup: expired tabu entries are removed.

Strengths and weaknesses

- ▶ Can move uphill to escape local minima.
- ▶ Memory prevents short search cycles.
- ▶ More expensive than Greedy and Swap-move.
- ▶ Robust compromise between quality and runtime.

Heuristic Methods: Comparison Summary

Method		Main idea	Best role	Main limitation
Greedy		Nearest feasible next node	Fast feasible seed	Myopic decisions
Swap-move Search	Local	Improve around current route	Fast exploitation	Local optimum trap
Random Walk		Random destroy-and-repair	Large-scale diversification	Stochastic quality variance
Tabu Search		Memory-guided moves	Robust escape from cycles	Higher runtime cost

Takeaway

The four methods form a pipeline: Greedy gives feasibility, Local Search exploits, Random Walk diversifies broadly, and Tabu Search balances improvement with memory-based escape.

- ▶ **Local benchmark:** random symmetric matrices, $N \in [5, 50]$, capacity $k = 3$.
- ▶ **Time limit:** 30 seconds per method in local experiments.
- ▶ **Online judge:** large hidden HUSTACK instances, $N \in [100, 500]$.
- ▶ **Metaheuristic limit:** 290 seconds to safely return before a 300-second hard limit.

Evaluation criteria

Feasibility, objective cost, runtime, and scalability.

Exact Local Benchmark Results

N	Greedy	Local	Tabu	CP-SAT	ILP-01	ILP-02	B&B
5	122	122	91	91	91	N/A	91
8	236	195	132	153	180	N/A	132
12	325	276	165	236	N/A	N/A	256
20	344	344	199	N/A	N/A	N/A	425
30	474	464	346	N/A	N/A	N/A	1305
50	782	741	676	N/A	N/A	N/A	2500

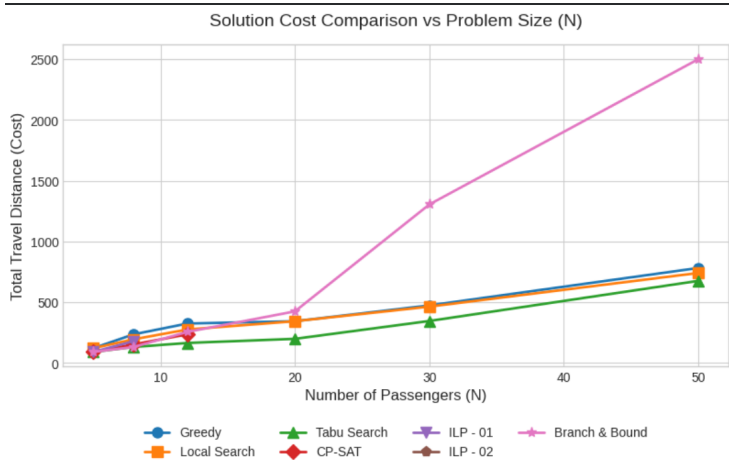
Interpretation: Bold entries mark best feasible costs; Tabu Search dominates medium/large tests, while exact solvers remain competitive only for small N .

Runtime to Best Solution

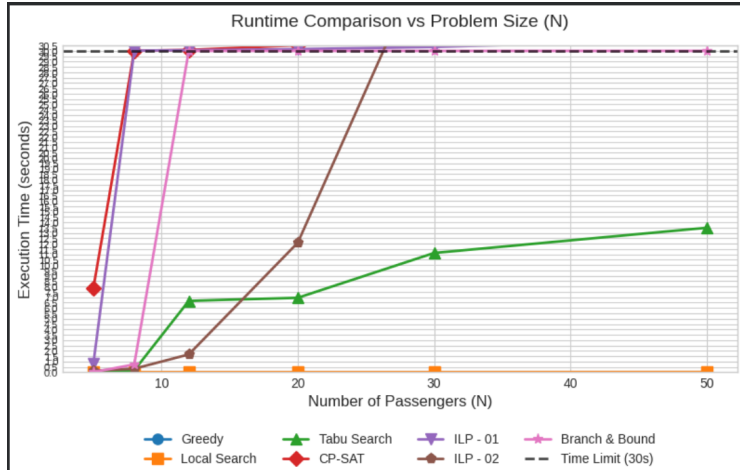
N	Greedy	Local	Tabu	CP-SAT	ILP-01	ILP-02	B&B
5	0.0000	0.0000	0.0083	7.8577	0.8037	0.1255	0.0091
8	0.0001	0.0003	0.1767	30.0175	30.0288	0.3275	0.7260
12	0.0001	0.0009	6.6621	30.0894	30.0680	1.6608	30.0000
20	0.0001	0.0001	6.9495	30.5319	30.1675	12.0933	30.0000
30	0.0003	0.0012	11.1309	31.4903	30.3690	40.6176	30.0000
50	0.0007	0.0121	13.4863	35.2536	31.0845	63.2985	30.0000

Interpretation: Greedy and Local Search are near-instant; Tabu spends more time for quality; exact methods quickly hit the time ceiling.

Local Benchmark: Solution Cost



Local Benchmark: Runtime Scalability



Online Judge Scores: Ascending Order

Method	Score	Type	Interpretation
Greedy Construction	474/500	Heuristic	Fast baseline but limited by myopic nearest-node choices.
Swap-move Local Search	482/500	Heuristic	Improves the greedy seed through feasible swap moves.
Tabu Search	484/500	Metaheuristic	Uses memory and aspiration to escape local traps.
Random Walk	486/500	Metaheuristic	Best observed score; relocation gives high diversification.

Observation

Diversification is essential at large scale; rigid local search can perform worse than the greedy route it improves from.

Result Screenshots: Lower Scores

Test Case						Trạng thái Evaluated
Tình điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	
✓	98	jury obj = 136 participant obj = 138	186	3,895	1	Đạt _ / 5 test case Điểm 482 Ngôn ngữ Python 3.8 Tổng thời gian chạy 339,386 (s) Thời gian tạo 23/06/2026 01:33:38 Bài tập Project 14 - CBUS Large Cuộc thi 20252 - Fundamental Optimization - Mini-Project
✓	100	Perfect	41	3,387	1	
✓	100	Perfect	43	3,383	1	
✓	91	jury obj = 10719 participant obj = 11751	290775	307,586	1	
✓	93	jury obj = 5936 participant obj = 6332	48,141	80,039	1	
5 hàng < > 1-5 của 5 >						

Greedy Construction: 474/500

Test Case						Trạng thái Evaluated
Tình điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác	
✓	95	jury obj = 136 participant obj = 142	71	3,867	1	Đạt _ / 5 test case Điểm 474 Ngôn ngữ Python 3.8 Tổng thời gian chạy 3,038 (s) Thời gian tạo 23/06/2026 01:31:28 Bài tập Project 14 - CBUS Large Cuộc thi 20252 - Fundamental Optimization - Mini-Project
✓	100	Perfect	39	3,258	1	
✓	100	Perfect	41	3,629	1	
✓	89	jury obj = 10759 participant obj = 12079	2,210	307,48	1	
✓	90	jury obj = 5936 participant obj = 6547	677	79,953	1	
5 hàng < > 1-5 của 5 >						

Swap-move Local Search: 482/500

Result Screenshots: Higher Scores

Test Case					
Tình điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác
✓	100	Perfect	289914	4,109	1
✓	100	Perfect	85	3,691	1
✓	100	Perfect	1367	3,68	1
✓	91	jury obj = 10753 participant obj = 11717	290032	307,844	1
✓	93	jury obj = 5936 participant obj = 6327	289950	80,258	1

Trạng thái
Evaluated

Đạt
... / 5 test case

Điểm
484

Ngôn ngữ
Python 3.8

Tổng thời gian chạy
871,348 (s)

Thời gian tạo
23/06/2026 01:26:30

Bài tập
Project 14 - CBUS Large

Cuộc thi
20252 - Fundamental Optimization - Mini-Project

5 hàng < > 1-5 của 5 >>

Tabu Search: 484/500

Test Case					
Tình điểm	Điểm	Thông báo	Thời gian chạy (ms)	Bộ nhớ (MB)	Thao tác
✓	100	Perfect	289958	4,066	1
✓	100	Perfect	289913	5,285	1
✓	100	Perfect	289837	3,445	1
✓	89	jury obj = 10662 participant obj = 11952	290147	307,816	1
✓	97	jury obj = 5933 participant obj = 6055	289909	80,289	1

Trạng thái
Evaluated

Đạt
... / 5 test case

Điểm
486

Ngôn ngữ
Python 3.8

Tổng thời gian chạy
1,449,764 (s)

Thời gian tạo
23/06/2026 01:33:48

Bài tập
Project 14 - CBUS Large

Cuộc thi
20252 - Fundamental Optimization - Mini-Project

5 hàng < > 1-5 của 5 >>

Random Walk: 486/500

Key Findings

1. **Exact methods are validation tools:** they are reliable for small cases but do not scale to operational sizes.
2. **Model structure matters:** arc-based ILP is more solver-friendly than sequence-based Big- M formulations.
3. **Greedy is a strong baseline:** fast feasibility is valuable under time pressure.
4. **Diversification wins at scale:** tabu memory and destroy-and-repair moves improve robustness.

Practical Deployment Pipeline

- Stage 1** → Greedy construction for immediate feasible route
- Stage 2** → Local search for fast exploitation
- Stage 3** → Tabu or random walk for diversification
- Stage 4** → Return best route before hard time limit

Recommended operational strategy

Use exact models as correctness references and deploy greedy–metaheuristic pipelines for large-scale routing performance.

Thank You

Questions and Discussion